Lecture 3
Bottom Up
Explicit Search

Armando Solar-Lezama

Idea

Generate programs one by one

- Generate & Test approach

Key issues

- In what order do you generate?
  - Influences performance *and* result quality
- How do you prune?
  - Essential for scalability
- How do you keep track of the remaining space?
  - Especially challenging in the context of pruning

Explicit search from grammars

Grammar describes how to generate program fragments from smaller program fragments

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

```
plist := set of all terminals
while(true){
   plist := grow(plist);
   forall( p in plist)
        if(isCorrect(p)){ return p; }
}

grow(plist){
 // return a list of all trees generated by
 // taking a non-terminal and adding
 // nodes in plist as children
}
```

Explicit search from grammars

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```

Grammar describes how to generate program fragments from smaller program fragments

Level 0

in [0] 0



Level 1

in	[0]	0	sort(in)	sort([0])	in[0,0]
[0][0,0]	in + in	in + [0]	[0] + [0]	[0] + in	rec(in)
rec([0])	firstZero(in)	firstZero([0])	len(in)	len([0])	0+1

Set grows very fast!

Large equivalence classes of equivalent programs

Identifying equivalent programs

Program equivalence is hard

- It is also unnecessary!

Observational Equivalence

- Are they equivalent wrt the inputs
  - easy to check efficiently
  - sufficient for the purpose of PBE
- Keep only the simplest one

```
plist := set of all terminals
while(true){
   plist := grow(plist);
   plist := elimEquvalents(plist);
   forall( p in plist)
        if(isCorrect(p)){ return p; }
}
```

Explicit search from grammars

Features:

- Search small programs before large programs
- Simple
- Works even with black-box language building blocks
  - no need to have source for sort or firstZero
    just need to be able to execute them
  - no need to know of any properties about them
    e.g. automatically ignores sort(sort(in)) without
    having to know that sort is idempotent
- Complexity depends on the size of the set of distinct programs
  - Copes well with symmetries

Explicit search from grammars

Limitations:

- Only scales to very small programs
- Unsuitable for programs with unknown constants
  - A single unknown 32-bit constant makes the problem intractable
- Hard to deal with context dependent semantics

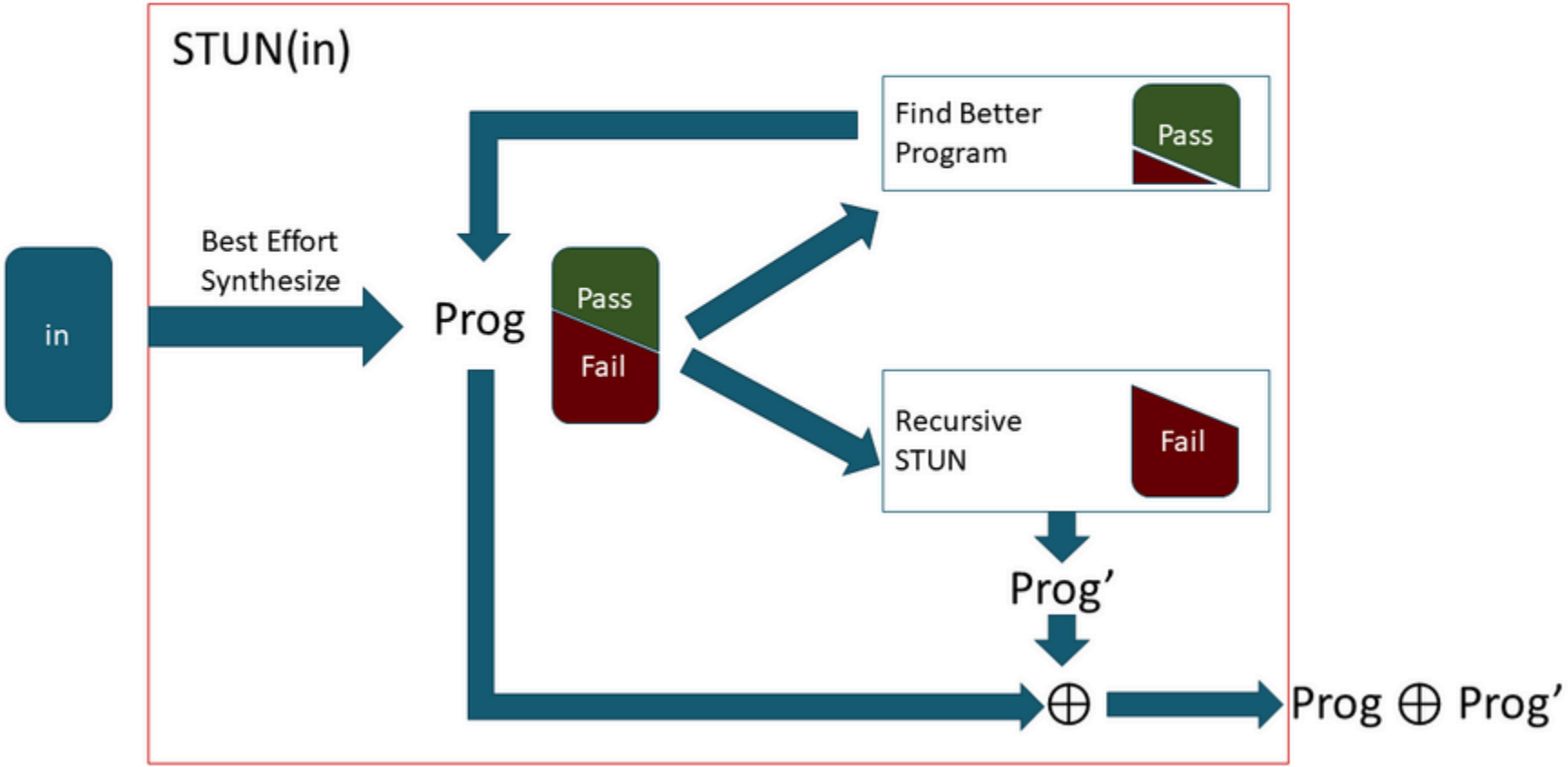
Example system:

- Recursive Program Synthesis [Albarghouthi et al., CAV 2013]

Synthesis Through Unification

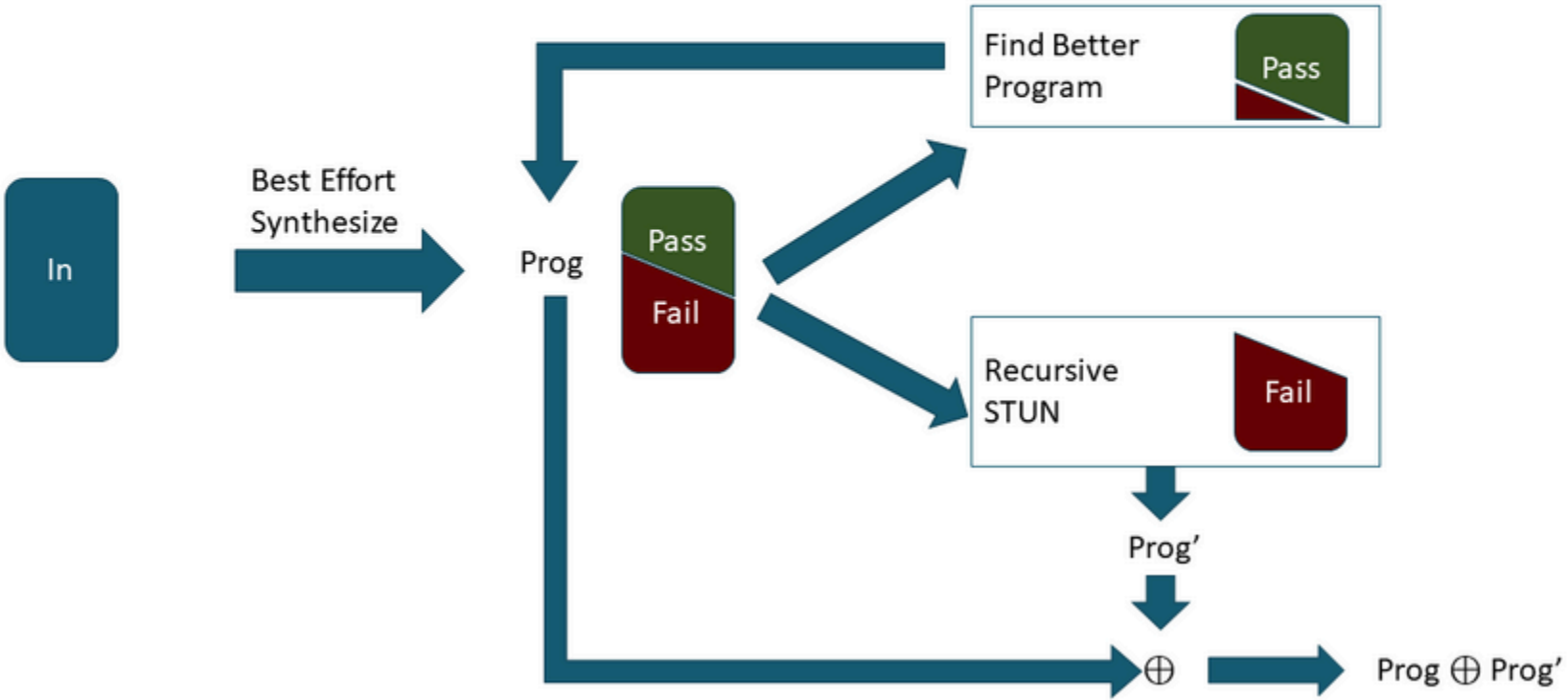
Idea: Solve many simpler problems, combine their solutions

STUN at a glance



STUN at a glance

(1,2) -> 1
(2,3) -> 2
(4,3) -> 3
(8,1) -> 8



Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
if(bexp) expr else expr
bexp = expr > expr | expr > 0

Level 1:	a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
if(bexp) expr else expr
bexp = expr > expr| expr > 0

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Eliminate Observationally equivalent ones

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
|if(bexp) expr else expr
bexp = expr > expr | expr > 0

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Identify an expression that works for a subset of the inputs

Grammar

```
expr = expr + expr
       | expr * expr
       | a | b | c
       | - expr
       | if(bexp) expr else expr
bexp = expr > expr | expr > 0
```

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(8, 11)* -1 => -11
(3,6)*4 => 12
(-3, 8) *4 => -12

Identify an expression that works for the rest of the inputs

Grammar

```
expr = expr + expr
       | expr * expr
       | a | b | c
       | - expr
       | if(bexp) expr else expr
bexp = expr > expr| expr > 0
```

Level 1:

a => [5,8,3,-3]	b=>[10,11,6,8]	C=>[3,-1,4,4]

a+a=[10,16,6,-6]	a+b=[15,19,9,5]	a+c=[8,7,7,1]	b+a=[15,19,9,5]	b+b=[20,22,12,16]	b+c=[13,10,10,12]	c+a=[8,7,7,1]
c+b=[13,10,10,12]	c+c=[6,-2,8,8]	a*a=[25,64,9,9]	a*b=[50,88,18,-48]	a*c=[15,-8,12,-12]	b*a=[50,88,18,-48]	b*b=[100,121,36,64]
b*c=[30,-11,24,32]	c*a=[15,-8,12,-12]	c*b=[30,-11,24,32]	c*c=[9,1,16,16]	-a=[-5,-8,-3,3]	-b=[-10,-11,-6,-8]	-c=[-3,1,-4,-4]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(3,6)*4 => 12
(-3, 8) *4 => -12

a*c P_1

(8, 11)* -1 => -11

b*c P_2

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
| if(bexp) expr else expr
bexp = expr > expr| expr > 0

a>a=[f,f,f,f]	a>b=[f,f,f,f]	a>c=[t,f,f,t]
b>a=[t,t,t,t]	b>b=[f,f,f,f]	b>c=[t,t,t,t]
c>a=[f,t,t,f]	c>b=[f,f,f,f]	c>c=[f,f,f,f]
a>0=[t,t,f,t]	b>0=[t,t,t,t]	c>0=[t,t,t,f]

Simple case: (a,b) * c => out

Examples

(5,10) * 3 => 15
(3,6)*4 => 12
(-3, 8) *4 => -12

a*c P_1

(8, 11)* -1 => -11

b*c P_2

Grammar

expr = expr + expr
| expr * expr
| a | b | c
| - expr
| if(bexp) expr else expr
bexp = expr > expr| expr > 0

a>a=[f,f,f,f]	a>b=[f,f,f,f]	a>c=[t,f,f,t]
b>a=[t,t,t,t]	b>b=[f,f,f,f]	b>c=[t,t,t,t]
c>a=[f,t,t,f]	c>b=[f,f,f,f]	c>c=[f,f,f,f]
a>0=[t,t,f,t]	b>0=[t,t,t,t]	c>0=[t,t,t,f]

$$P_1 \oplus P_2 = if\,(c > 0)\; a * c\; else\;\; b * c$$

Why was this better?

What if there is no top level branch?

The same approach generalizes for other domains without branches

Grammar

```
var = a | b | c
expr = expr + expr
       | expr * expr
       | var
       | - expr
       | if(bexp) var else var
bexp = expr > expr| expr > 0
```

Antiunification

Given two expressions $expr_1$, $expr_2$ find an expression $expr^*$ with some free variables v_i such that

- $expr_1 = expr^*[v_i \rightarrow e_i^1]$
- $expr_2 = expr^*[v_i \rightarrow e_i^2]$

$Antiunify(expr_1, expr_2) = expr^*$

In general, you want to maximally expose sharing between $expr_1$ and $expr_2$.

- You want to avoid just making $expr^* = v$

What if there is no top level branch?

Examples

(5,10) * 3 => 15 (3,6)*4 => 12 (-3, 8) *4 => -12	a*c	P_1
(8, 11)* -1 => -11	b*c	P_2

Antiunify(a*c, b*c) = v*c

Find an expression v such that

(5,10) * 3 => v=5 (3,6)*4 => v=3 (-3, 8) *4 => v=-3	(8, 11)* -1 => v=11

Grammar

var = a | b | c
expr = expr + expr
| expr * expr
| var
| - expr
| if(bexp) var else var
bexp = expr > expr| expr > 0

What if there is no top level branch?

Examples

(5,10) * 3 => 15 (3,6)*4 => 12 (-3, 8) *4 => -12	a*c	P_1
(8, 11)* -1 => -11	b*c	P_2

Grammar

var = a | b | c
expr = expr + expr
| expr * expr
| var
| - expr
| if(bexp) var else var
bexp = expr > expr| expr > 0

Antiunify(a*c, b*c) = v*c

Find an expression v such that

(5,10) * 3 => v=5 (3,6)*4 => v=3 (-3, 8) *4 => v=-3	(8, 11)* -1 => v=11	v=if(c>0) a else b

STUN Summary

STUN provides a general framework for breaking down a global search into a series of local searches.

Last time

Explicit search

- With pruning for observational equivalence

Alternative interpretation of the algorithm

The algorithm from last time can be understood in terms of partial orders

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



Search for smallest element ≤ 

Binary search



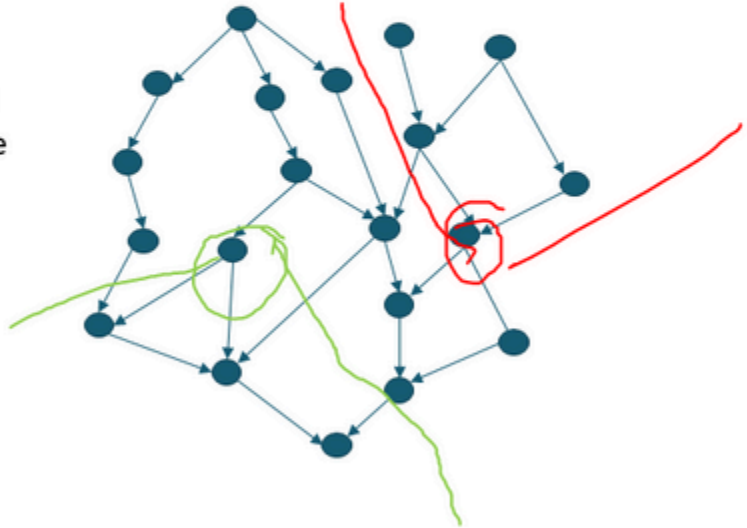
Search for smallest element ≤ 

Partial order for search

$p_1 \sqsubseteq p2$



If p_1 is bad, all $\{p_2 | \ p_1 \sqsubseteq p_2\}$ are bad
If $p_1 \sqsubseteq p_2$ then p_2 is always preferable



Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

Eliminate by observational equivalence

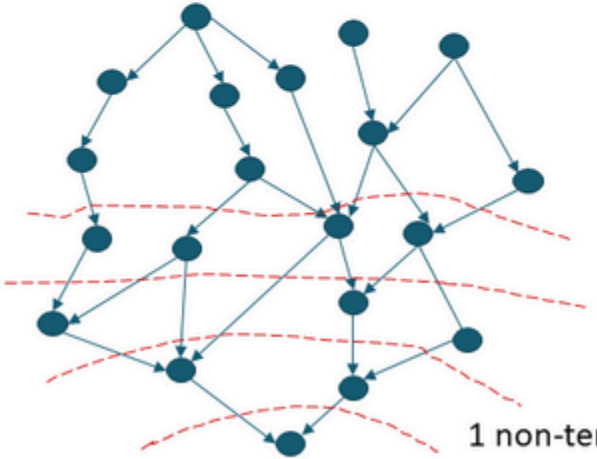
If p_i is bad, we can eliminate all $P = \{p \mid p_i \sqsubseteq p\}$

- Bad == it's equivalent to something else I am already considering

Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```



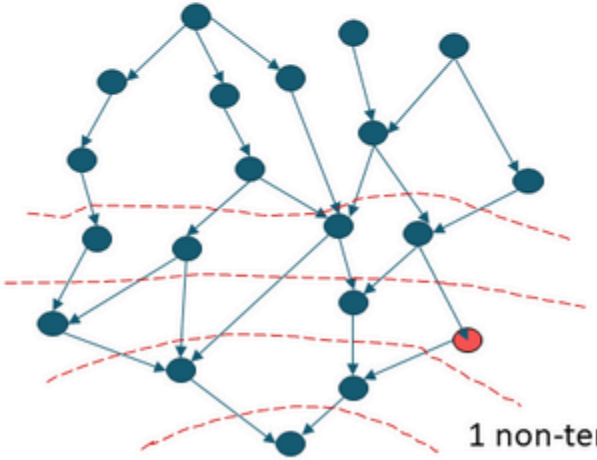
1 non-terminal connecting terminals

Terminals

Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```



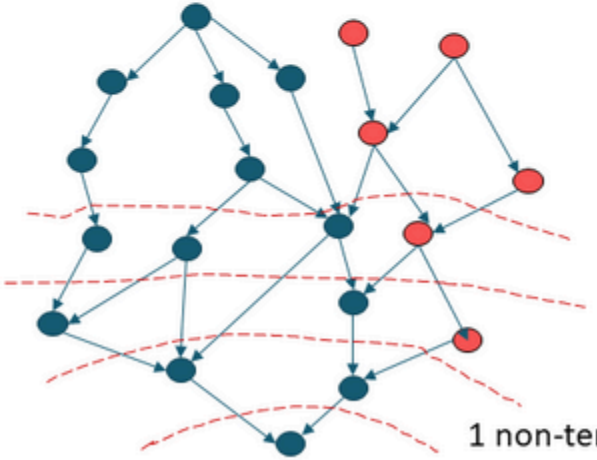
1 non-terminal connecting terminals

Terminals

Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```



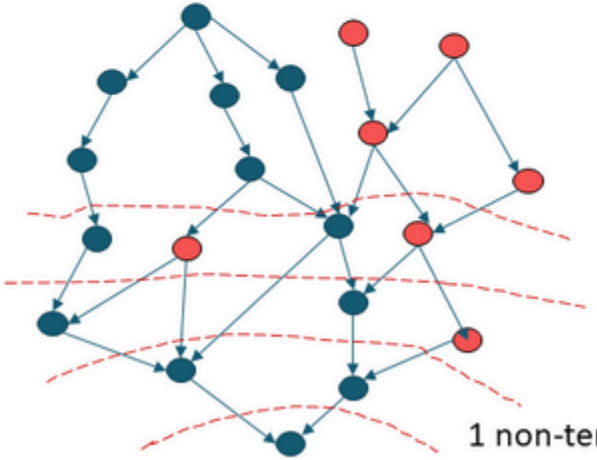
1 non-terminal connecting terminals

Terminals

Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```



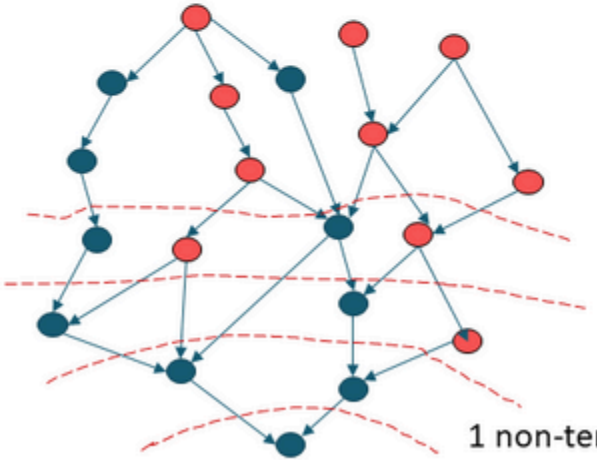
1 non-terminal connecting terminals

Terminals

Partial order for grammar

$p_1 \sqsubseteq p_2$ iff p_1 is a subprogram of p_2

```
lstExpr := sort(lstExpr)
           lstExpr[intExpr,intExpr]
           lstExpr + lstExpr
           recursive(lstExpr)
           [0]
           in
intExpr := firstZero(lstExpr)
           len(lstExpr)
           0
           intExpr + 1
```



1 non-terminal connecting terminals

Terminals

Version spaces

Programming by demonstration using version space algebra

Lau, Wolfman, Domingos, Weld, 2001

Version Space Formulation

Hypothesis space H

- Space of possible functions $In \rightarrow Out$

Version Space $VS_{H,D} \subseteq H$

- H is the original hypothesis space
- D is a set of examples i_j, o_j
- $h \in VS_{H,D} \Leftrightarrow \forall\, i, o \in D \;\; h(i) = \mathrm{o}$

Hypothesis space provides *restriction bias*

- Defines what functions one is allowed to consider
- *Preference bias* needs to be provided independently

Partial Ordering of hypothesis

Partial order $h_1 \sqsubseteq h_2$

- h_2 is "better" than h_1

Ex: For boolean hypothesis

- "better" == more general
- $h_1 \sqsubseteq h_2 \Leftrightarrow (h_1 \Rightarrow h_2)$

For booleans, VS forms a lattice

Partial Orders

Set P

Partial order $\leq$ such that $\forall x,y,z \in P$

- $x \leq x$ (reflexive)
- $x \leq y$ and $y \leq x$ implies $x = y$ (asymmetric)
- $x \leq y$ and $y \leq z$ implies $x \leq z$ (transitive)

Can use partial order to define

- Upper and lower bounds
- Least upper bound
- Greatest lower bound

Upper Bounds

If $S \subseteq P$ then

- $x \in P$ is an upper bound of S if $\forall y \in S.\ y \leq x$
- $x \in P$ is the least upper bound of S if
  - x is an upper bound of S, and
  - $x \leq y$ for all upper bounds y of S
- $\vee$ - join, least upper bound, lub, supremum, sup
  - $\vee S$ is the least upper bound of S
  - $x \vee y$ is the least upper bound of {x,y}
- Often written as $\sqcup$ as well

Lower Bounds

If $S \subseteq P$ then

- $x \in P$ is a lower bound of S if $\forall y \in S.\ x \leq y$
- $x \in P$ is the greatest lower bound of S if
  - x is a lower bound of S, and
  - $y \leq x$ for all lower bounds y of S
- $\wedge$ - meet, greatest lower bound, glb, infimum, inf
  - $\wedge$ S is the greatest lower bound of S
  - $x \wedge y$ is the greatest lower bound of {x,y}
- Often written as $\sqcap$ as well

Lattices

If $x \wedge y$ and $x \vee y$ exist for all $x,y \in P$
then P is a lattice

If $\wedge S$ and $\vee S$ exist for all $S \subseteq P$
then P is a complete lattice

All finite lattices are complete

Example of a lattice that is not complete

- Integers I
- For any $x, y \in I$, $x \vee y = \max(x,y)$, $x \wedge y = \min(x,y)$
- But $\vee I$ and $\wedge I$ do not exist
- $I \cup \{+\infty, -\infty\}$ is a complete lattice

Partial Ordering of hypothesis

Partial order $h_1 \sqsubseteq h_2$

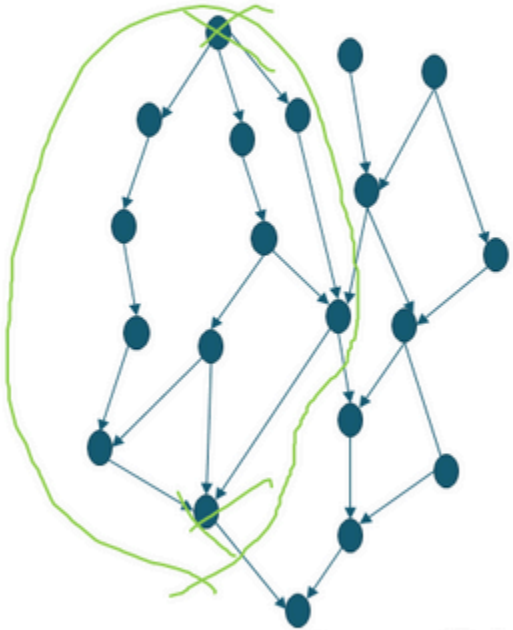
- h_2 is "better" than h_1

Ex: For boolean hypothesis

- "better" == more general
- $h_1 \sqsubseteq h_2 \Leftrightarrow (h_1 \Rightarrow h_2)$

For booleans, VS forms a lattice

- $h_1, h_2 \in VS \Rightarrow h_1 \sqcap h_2 = h_1 \wedge h_2 \in VS$



Most specific hypothesis that satisfies the observations

Boundary set representable

You can represent a VS by the pair (G,S) where

- G is most general hypothesis (i.e. $\top$)
- S is the most specific (i.e. $\bot$)

Applies in general when hypothesis space is partially ordered and version space is a lattice

Update

$$U(VS, d) = \{p \in VS \mid p(i) = o \; where \; d = (i, o)\}$$

- Subset of a version space satisfying a new example d

Ex: For boolean HS

- VS=(G,S)
- If $d = (i, true)$

  $U(VS, d) = (G, S \vee \lambda x. if \; x = i \; then \; true \; else \; false)$
- If $d = (i, false)$

  $U(VS, d) = (G \wedge \lambda x. if \; x = i \; then \; false \; else \; true, S)$

Example: FindSuffix

FS_T: next occurrence of T.

Idea

If your hypothesis space is partially ordered and your VS are boundary set representable, you can represent and search very efficiently

If they are not?

Break them down into simpler hypothesis spaces!

Union

$$VS_{H_1 D} \cup VS_{H_2 D} = VS_{H_1 \cup H_2\ D}$$

Join

$$VS_{H_1D_1} \bowtie VS_{H_2D_2} = \{\langle h_1, h_2\rangle \mid h_1 \in VS_{H_1D_1}, h_2 \in VS_{H_2D_2}, C(\langle h_1, h_2\rangle, D)\}$$

- Where $D_1 = \{d_1^i\}_{i=0..n}$ and $D_2 = \{d_2^i\}_{i=0..n}$ and $D = \{\langle d_1^i, d_2^i\rangle\}_{i=0..n}$
- $C(\langle h_1, h_2\rangle, D)$ means that $\langle h_1, h_2\rangle$ is consistent with the input output pairs in D

What does $\langle h_1, h_2\rangle$ mean? What about $\langle d_1, d_2\rangle$?

- Pair
- Composition $\langle h_1, h_2\rangle = h_1 \circ h_2$ and $\langle d_1, d_2\rangle = (d_1.in, d_2.out)$

Independent join: C is unnecessary

- It's a property of $\langle .,. \rangle$
- True for pair, not for composition

Transform

$$VS_2 = transform_{\tau_i,\tau_o}(VS_1) \text{ iff}$$
$$\forall g \in VS_2 \; \exists f \in VS_1 \; s.t. \; g = \tau_i \circ f \circ \tau_o$$

SMARTedit version space

